

Solution Documentation: Intelligent Candidate Discovery Engine

1. Solution Overview (The Big Picture)

The Intelligent Candidate Discovery Engine is an AI-powered recruiter. Its goal is to sift through a massive talent pool of 100,000 candidates and output the **absolute best 100 people** for a specific "Senior AI Engineer" role.

It must do this quickly (under 5 minutes on a standard computer) and accurately.

Why is this different from normal hiring tools? Normal tools just count keywords. If a job asks for "AI," they find the person who typed "AI" the most times. But this dataset is full of traps. It contains "keyword stuffers" (people who pasted buzzwords but have no real experience) and "honeypots" (fake, impossible profiles).

Our solution acts like a seasoned human recruiter. It reads between the lines to find "shippers"—engineers who actually build and deploy real products—while filtering out "ghosts" (candidates who never reply to messages) and purely academic researchers.

2. Dataset Description (What We Are Working With)

We are processing a massive file (`candidates.jsonl`) containing 100,000 profiles. Each profile has several parts:

- **Static Profile:** Basic facts like name, location (e.g., Pune/Noida), job title, and total years of experience.
- **Career History:** Where they worked, for how long, and what they actually did (written in plain text).
- **Skills:** Technical skills, how long they've used them (in months), and how many endorsements they have.
- **Redrob Signals:** A goldmine of 23 behavioral data points. This tells us if they are actually hireable. It includes their notice period, how fast they reply to recruiters, their GitHub activity score, and when they last logged in.

⚠ **The Traps We Must Avoid:** The organizers explicitly warned us that the dataset contains:

- **Honeypots:** About 80 fake profiles with impossible combinations of skills and experience. If more than 10% of our top 100 are honeypots, we get disqualified.
- **Keyword Stuffers:** Candidates with every AI buzzword but irrelevant job titles (like "Marketing Manager").
- **Behavioral Twins:** Candidates with identical skills, but one is active and the other hasn't logged in for a year. We must pick the active one.

3. Points for Feature Engineering (What Data We Actually Use)

We cannot use all the data because we have a strict 5-minute time limit. We must only extract the most powerful "features" (clues) that prove a candidate matches the Job Description (JD):

1. **Semantic Context (The "Vibe" Check):** We look at their summary and experience to see if they talk about *building* systems, not just studying them.
2. **Exact Keywords (The Hard Skills):** We check if they specifically mention necessary tools like "FAISS", "Pinecone", "NDCG", or "Python".
3. **True Skill Weight:** Instead of just trusting they know "Machine Learning," we calculate: $\text{Months of Experience} \times \text{Endorsements}$. This proves they actually used it.
4. **Career Velocity:** We check if they are "title-chasers" (hopping jobs every year for a better title) or if they have steady, meaningful promotions.
5. **Behavioral Intent (The Ghost Penalty):** We heavily penalize candidates with >30 day notice periods, poor recruiter response rates, or those who haven't logged in recently.
6. **The "Pure Services" Penalty:** As requested by the JD, we flag candidates who have *only* worked at IT consulting firms (like TCS or Infosys) with zero product-company experience.

4. ML Components (The Math, Explained Simply)

Our system uses two main concepts: **Hybrid Search** and **Learning-to-Rank**.

Concept 1: Hybrid Retrieval (Finding the Top Candidates)

Imagine trying to find a needle in a haystack. We use two different tools at the same time to narrow down the 100,000 candidates:

- **Tool A: BM25 (The Strict Librarian):** This is a math formula that looks for *exact* words. If the job requires "Python", BM25 ensures the candidate actually wrote "Python". It is great for hard facts.
- **Tool B: FAISS (The Matchmaker):** This is an AI tool that understands concepts (Vector Embeddings). If the job asks for a "Vector Database," and the candidate wrote "Pinecone" (a type of vector database), FAISS knows they mean the same thing, even though the words are different.

Formula: We combine their scores. $\text{Hybrid_Score} = (\text{BM25} + \text{FAISS})$.

Concept 2: LightGBM (The Head Judge)

Once Hybrid Search finds the top 5,000 closest matches, we hand them over to **LightGBM**. Think of LightGBM as a master judge. It looks at the `Hybrid_Score`, adds in the `Behavioral Signals` (like their response rate), looks at their `Career Velocity`, and calculates one final, perfect "Hireability Score" from 1 to 100.

5. System Architecture (The Step-by-Step Flow)

Based on the architecture diagram, our system runs in three distinct phases.

Section 1: Data Pre-processing (The Offline Prep)

This happens on our laptops before the competition clock starts.

- **Input:** The massive 100k Dataset.
- **Process:** We clean up informal text (e.g., changing "did backend" to "Backend Architecture"). We build the "Strict Librarian" (BM25) index and the "Matchmaker" (FAISS) index.
- **Output:** We save these indexes as files so they are ready to be loaded instantly.

Section 2: The Ranking Funnel (The 5-Minute Race)

This is the code that runs when the judges test our system.

- **Stage 1 (Hard Disqualification):** The "Gatekeeper" instantly deletes obvious honeypots, candidates with zero verifications, or extreme domain mismatches. *This drops ~40% of the candidates instantly, saving us time.*
- **Stage 2 (Hybrid Retrieval):** The system asks BM25 and FAISS to find the closest matches. We take only the **Top 5,000** candidates forward.
- **Stage 3 (Machine Learning Scoring):** The Top 5,000 candidates are graded by the LightGBM judge, combining their technical skills with their behavioral signals (response rates, active days).
- **Stage 4:** The list is sorted, and we chop off everyone except the absolute **Top 100**.

Section 3: Justification Generation

- **Process:** Because we cannot use ChatGPT or external internet APIs during the test, we use Python string templates to write a sentence for each of the top 100 candidates. (e.g., *"Candidate has 6 years of product ML experience, 90% response rate, and verified GitHub."*)

6. System Components Breakdown (The Glossary)

- **Normalizer:** A text-cleaning tool that fixes messy resumes before the AI reads them.
- **BM25 Engine:** The search engine for exact keywords.
- **Vector Engine (FAISS):** The AI search engine for conceptual meaning.
- **Gatekeeper:** A simple true/false checklist that deletes fake or terrible profiles instantly.
- **LightGBM Scorer:** The Machine Learning algorithm that combines all the data to create the final ranking score.
- **NLG Generator:** A simple piece of code that glues data together to write human-readable sentences explaining *why* someone was chosen.

